

MCDI500

PROGRAMACIÓN PARA LA CIENCIA DE DATOS

AVANCE DEL PROYECTO FASE 3 – SEMANA 2

Nacimientos en Chile entre 2020 y 2023

Integrantes: Cristian Hurtado, Jorge Altamirano, Renzo Vilchez y Luis Paredes

Docente: Dr. Omar Salinas Silva

Curso: MCDI500 – Programación para la Ciencia de Datos

Fecha: 29 de septiembre de 2026

ÍNDICE

1. Introducción y alcance del avance	3
2. Codificación funcional y arquitectura básica	3
3. Preprocesamiento y transformación del dataset	3
4. Validación técnica y verificación	4
5. Eficiencia y optimización	4
6. Diseño estructurado y modularidad	4
7. Implementación modular y programación orientada a objetos	4
8. Documentación de arquitectura y repositorio	5
9. Notebook ejecutado y reproducibilidad	5
10. Conclusiones del avance	5
Referencias	6

1. Introducción y alcance del avance

Este informe documenta el prototipo algorítmico de la Fase 3. El proyecto analiza los nacidos vivos inscritos en Chile entre 2020 y 2023 a partir de datos abiertos del DEIS/MINSAL. La Fase 2 dejó un conjunto preparado de 735.611 filas; esta entrega evalúa cómo procesarlo de manera verificable y eficiente, sin rehacer la limpieza ni cambiar las preguntas descriptivas. Una fila representa un nacido vivo inscrito, no necesariamente una madre diferente.

El avance contiene un notebook ejecutado, funciones separadas en un módulo, verificaciones de equivalencia, tres repeticiones por medición y una transformación nominal requerida por el docente. El diseño se ajusta al carácter de prototipo descrito en la guía de la Formativa 3 (Salinas Silva, 2026). La consigna de la Formativa 2 plantea comparar programación estructurada y recursiva, tiempo y memoria (Universidad Andrés Bello, 2026). En su intervención inicial del 29 de septiembre, Cristian Hurtado Cabezas distinguió el procesamiento tabular de la recursividad sobre metadatos anidados y propuso verificar resultados equivalentes antes de comparar tiempo y memoria. Este avance aplica esos criterios.

2. Codificación funcional y arquitectura básica

El archivo `F3/src/f3_algoritmos.py` contiene funciones de lectura, cálculo y medición. `cargar_preparado` verifica la salida de F2; `proporcion_bucle` y `proporcion_groupby` calculan el mismo indicador regional; `medir_tiempos` y `medir_memoria_pico` observan costos; `leer_original` permite comparar lecturas del CSV. La clase `AnalizadorNacimientos`, propuesta por Jorge Altamirano, reúne la carga del conjunto preparado y los dos cálculos regionales, reutilizando esas funciones.

El notebook *F3/F3_Avance_Semana2.ipynb* coordina las funciones, muestra resultados intermedios y comprueba las invariantes. Esta separación evita ocultar la lógica en una única celda y permite reemplazar una implementación sin alterar las demás. Las funciones del módulo devuelven valores o generan errores claros, en lugar de modificar los archivos fuente.

3. Preprocesamiento y transformación del dataset

La entrada analítica es *data/processed/nacimientos_preparados_propuesta.csv*, generada en F2. Se seleccionan *REGION_RESIDENCIA* y *madre_menor_20* porque bastan para el indicador. En F2 ya se había recodificado la edad materna para marcar el grupo menor de 20 años, construido un período mensual a partir de año y mes, documentado la validez técnica de talla y comparado cuatro escenarios de imputación. Este avance no vuelve a estimar esos valores.

La región es nominal: sus códigos enteros no expresan orden ni distancia. Para la demostración de codificación se divide el conjunto en entrenamiento y prueba con semilla 42 y proporción 80/20. *OneHotEncoder* aprende las 16 categorías observadas solo en entrenamiento y aplica las mismas 16 columnas a prueba. El notebook muestra una tabla de cuatro ejemplos reales: cada región activa una sola columna indicadora. Una categoría hipotética no observada conserva el esquema y recibe ceros; esa fila no puede invertirse únicamente desde la matriz, por lo que se conserva el conjunto con la región original y la correspondencia de filas. La documentación oficial describe este comportamiento de *handle_unknown='ignore'* (scikit-learn developers, s. f.; Pedregosa et al., 2011).

La talla permanece en centímetros. Este prototipo no usa métodos basados en distancias y, por tanto, no necesita escalarla. La matriz codificada es un objeto separado de la versión legible utilizada para tablas y gráficos.

4. Validación técnica y verificación

La ejecución completa comprueba que las columnas requeridas no tienen nulos y que los códigos regionales están entre 1 y 16. Las versiones con bucle y *groupby* entregaron los mismos conteos y porcentajes por región; la suma de nacimientos regionales coincide con las 735.611 filas y la suma de las marcas maternas coincide con el total de la columna original.

En la comparación de lectura se constató igualdad fila por fila de las seis columnas pertinentes. Para la codificación se verificaron igual número y nombre de columnas en entrenamiento y prueba, una única categoría activa por fila conocida y ausencia de columna nueva ante la categoría hipotética 99. La función recursiva *aplanar* de F2 también se comprobó con metadatos anidados de dos niveles. Estas verificaciones corresponden a evidencia básica, no a una batería exhaustiva de pruebas.

5. Eficiencia y optimización

Se usó *time.perf_counter()* y se conservó el menor tiempo de tres repeticiones con el conjunto completo, conforme a la guía docente (Salinas Silva, 2026; Python Software Foundation, s. f.). Los tiempos describen esta ejecución en Python 3.13.5, pandas 3.0.5, NumPy 2.5.2 y scikit-learn 1.9.0; pueden variar en otro equipo.

Operación	Versión A	Versión B	Verificación
Indicador regional	Bucle: 0.1423 s	<i>groupby</i> : 0.0090 s	Mismos conteos
Lectura del CSV	25 columnas: 0.9710 s	6 columnas: 0.5450 s	Mismas 6 columnas
Memoria del DataFrame	408.08 MB	127.30 MB	Objetos cargados

En esta medición, *groupby* fue 15.86 veces más rápido que el bucle. La lectura selectiva fue 1.78 veces más rápida y el DataFrame resultante ocupó 3.21 veces menos memoria. Se adopta *groupby* para el indicador y *usecols* para lecturas que solo necesiten las seis variables. El parámetro *usecols* evita materializar columnas no usadas, según la documentación de pandas (The pandas development team, s. f.). La memoria informada es la ocupación profunda del DataFrame; no representa el pico total del proceso.

6. Diseño estructurado y modularidad

El diseño actual usa funciones porque cada cálculo es independiente. Se reutiliza *aplanar* desde *src/f2_utilidades.py*: se aplicó a los metadatos reales de esta ejecución, que incluyen versiones, tiempos y parámetros de codificación. Su caso recursivo descende en diccionarios anidados y su caso base registra un valor simple. No se emplea recursión sobre las filas de nacimientos, donde un recorrido o una agrupación son más adecuados. Esta distinción responde al criterio de pertinencia explicado en la guía docente (Salinas Silva, 2026).

7. Implementación modular y programación orientada a objetos

Se incorporó *AnalizadorNacimientos* para organizar la lectura del conjunto preparado y las dos formas de calcular el indicador regional. Sus métodos exigen que los datos se carguen antes de calcular. El notebook comprueba que el bucle y *groupby* producen los mismos resultados y mide el tiempo de cada alternativa. La clase organiza el uso de esos cálculos; por sí sola no demuestra una mejora de velocidad.

Las funciones independientes permanecen disponibles para mediciones sobre distintos tamaños de entrada.

8. Documentación de arquitectura y repositorio

Ruta	Responsabilidad
data/processed/	Conjunto preparado y legible de F2; entrada analítica de F3.
src/f2_utilidades.py	Función recursiva reutilizada para metadatos.
F3/src/f3_algoritmos.py	Lectura, clase regional, dos cálculos equivalentes y mediciones.
F3/F3_Avance_Semana2.ipynb	Orquestación, pruebas visibles, interpretación y codificación.
F3/resultados/	JSON pequeños de tiempos, versiones y vocabulario nominal.
README.md y F3/README.md	Ejecución, alcance, dependencias y decisiones técnicas.

La arquitectura conserva F2 como referencia y añade F3 sin copiar sus CSV. El avance está publicado en una rama del repositorio del grupo para revisión mediante un *pull request*. Su integración a *main* queda sujeta a la revisión del equipo.

9. Notebook ejecutado y reproducibilidad

El cuaderno se ejecutó completo, en orden, con nueve celdas de código y salidas visibles. Registra versiones efectivas y rutas, verifica equivalencia antes de interpretar tiempos, y escribe *mediciones.json* y *parametros_codificacion.json*. Para reproducirlo se abre la raíz del repositorio, se crea el entorno virtual con *requirements.txt*, se selecciona ese kernel y se ejecutan todas las celdas tras reiniciar. *F3/README.md* contiene las instrucciones concretas. F1, F2 y F3 se ejecutaron sin errores con las versiones declaradas en el archivo principal de dependencias.

10. Conclusiones del avance

El prototipo produce el mismo indicador con ambas implementaciones y demuestra una ventaja temporal de la agrupación de *pandas* en el conjunto completo. La lectura selectiva reduce el tiempo y la memoria del objeto cargado sin cambiar las seis variables utilizadas. La codificación nominal mantiene separados entrenamiento y prueba y conserva los datos interpretables para etapas posteriores. El módulo de funciones y las verificaciones visibles establecen una base para ampliar el proyecto sin rehacer F2 y usar una clase para reunir la lectura preparada y los dos cálculos regionales.

Referencias

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
<https://www.jmlr.org/papers/v12/pedregosa11a.html>
- Python Software Foundation. (s. f.). *time* — Time access and conversions.
<https://docs.python.org/3/library/time.html>
- Salinas Silva, O. (2026). *Guía de apoyo: Evaluación Formativa 3, Fase 3* [Material de curso]. Universidad Andrés Bello.
- scikit-learn developers. (s. f.). *OneHotEncoder*.
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>
- The pandas development team. (s. f.). *pandas.read_csv*.
https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
- Universidad Andrés Bello. (2026). *Formativa 2: Decisiones de diseño algorítmico* [Consigna de foro]. MCDI500.